

Endterm Learning Report

Data Structures and Algorithms

This report reflects how my approach to programming problems has developed during the course. At the beginning, I mainly focused on obtaining the correct output and often considered a problem complete once it passed the sample test cases. While attempting more questions, I began encountering Time Limit Exceeded errors, invalid indexing, repeated calculations, unnecessary memory usage, and recursive functions that did not stop correctly. These experiences made me realise that a correct answer is only one part of a complete solution. I am now reasonably comfortable with several easy-level questions, but medium-level problems are still challenging when the required pattern is not immediately visible. In such situations, I have sometimes used ChatGPT to understand the basic intuition, identify a suitable technique, or see how a larger problem can be divided into smaller parts. I then try to trace the algorithm manually, rewrite it in my own style, and understand its complexity and correctness rather than directly copying a finished answer. This has helped me build a better foundation, although recognising medium-level patterns independently is still something I am working on.

I first started Dynamic Programming with three basic problems: Climbing Stairs, Min Cost Climbing Stairs, and House Robber. After that, I added on more DP topics such as subsequences, strings, stocks, grid DP, and some harder pattern-based problems. I also began graph topics such as DFS, BFS, topological sorting, shortest path algorithms, and Union-Find. These newer topics were definitely harder than the earlier array questions because the solution was not just about writing loops, but about defining the right state, graph representation, queue content, or invariant.

A major learning from the newer topics was that algorithms are not always used in their plain textbook form. Many questions required creatively manipulating a known idea. For example, BFS could become multi-source BFS, Dijkstra could be adjusted with extra state, a DP table could represent two strings instead of one array, and Union-Find could group emails, stones, or components depending on the problem. This was challenging, but it made the topics feel more connected and less like separate formulas.

Rather than only listing the topics covered, this report discusses the ideas, mistakes, mathematical reasoning, correctness proofs, and practical observations that contributed most to my learning.

Name: Aditya Murukate

Roll Number: 24B2245

Course: Data Structures and Algorithms

Submission: Endterm Learning Report

July 2026

Contents

1	Introduction and Development of My Approach	4
1.1	Understanding an Explanation Versus Producing a Solution	4
1.2	Correctness Is Not the Final Step	4
2	Multiple Solutions and Complexity Analysis	6
2.1	Example: Contains Duplicate	6
2.2	Sorting-Based Solution	6
2.3	Set-Based Solution	7
2.4	Time–Memory Trade-Off	7
2.5	Common Complexity Classes	7
3	Array Traversal, Frequency Counting, and Prefix Techniques	9
3.1	Maximum Consecutive Ones	9
3.2	Majority Element and Cancellation	9
3.3	Kadane’s Algorithm	10
3.4	Product of Array Except Self	10
3.5	Prefix Sums	11
3.6	Prefix Remainders	12
4	Sliding Windows and Two-Pointer Methods	13
4.1	General Sliding-Window Structure	13
4.2	Why the Complexity Is Linear	13
4.3	Longest Substring Without Repeated Characters	13
4.4	Fixed-Size Sliding Windows	14
4.5	Two Pointers on Sorted Data	14
4.6	Three Sum	14
5	Sorting Algorithms and Binary Search	16
5.1	Bubble Sort	16
5.2	Selection Sort	16
5.3	Insertion Sort	16
5.4	Merge Sort	17
5.5	Binary Search	18
5.6	Binary Search on the Answer	18
6	Stacks, Queues, Deques, and Monotonic Structures	19
6.1	Valid Parentheses	19
6.2	Queues	19
6.3	Monotonic Stack	19
6.4	Monotonic Deque	20

7	Recursion and Backtracking	21
7.1	Factorial and Induction	21
7.2	Recursion Depth	21
7.3	Backtracking	21
7.4	Generating Subsets	22
7.5	Generate Parentheses and Pruning	22
8	Introduction to Dynamic Programming	24
8.1	Why Dynamic Programming Is Useful	24
8.2	Memoisation and Tabulation	24
8.3	Climbing Stairs	24
8.4	Min Cost Climbing Stairs	25
8.5	House Robber	26
8.6	What I Learned from Basic DP	27
9	Further Dynamic Programming: Subsequences, Strings, Stocks, and Grids	28
9.1	DP on Subsequences	28
9.2	Coin Change and Counting Versus Minimising	29
9.3	DP on Strings	29
9.4	Edit Distance and Thinking in Operations	30
9.5	DP on Stocks	30
9.6	LIS and Pattern-Based DP	31
9.7	Grid DP and Matrix-Based Problems	31
9.8	Main DP Learning	31
10	Graph Algorithms, BFS, DFS, and Shortest Paths	33
10.1	DFS and Connected Components	33
10.2	BFS and Level-by-Level Thinking	33
10.3	Grid Problems as Graphs	34
10.4	Topological Sorting	35
10.5	Shortest Path Algorithms	35
10.6	Manipulating Shortest Path Algorithms	36
10.7	Floyd–Warshall	37
11	Union-Find, MST Ideas, and Algorithm Adaptation	38
11.1	Basic Union-Find	38
11.2	Connected Components and Network Problems	38
11.3	Accounts Merge	39
11.4	Most Stones Removed	39
11.5	Kruskal’s and Prim’s Algorithms	39
11.6	Learning to Manipulate Algorithms	39

12 Binary Trees and Binary Search Trees	41
12.1 Tree Traversals	41
12.2 Maximum Depth	41
12.3 Level-Order Traversal	42
12.4 Binary Search Trees	42
12.5 Why Inorder Traversal Is Sorted	43
13 Greedy Algorithms and Heaps	44
13.1 Assigning Cookies	44
13.2 Interval Scheduling	44
13.3 Jump Game	44
13.4 Heaps	45
13.5 Finding the k -th Largest Element	45
14 Implementation Errors, Time Limits, and Memory	46
14.1 Index Errors	46
14.2 Negative Indices	46
14.3 Empty Containers	46
14.4 None in Trees	46
14.5 Python List Slicing	46
14.6 List Membership	47
14.7 Copying and Shared Mutation	47
14.8 Time Limit Exceeded	47
14.9 Memory Limit Exceeded	47
15 Correctness Proofs and Invariants	48
15.1 Insertion Sort Invariant	48
15.2 Binary Search Invariant	48
15.3 Sliding-Window Invariant	48
15.4 Heap Invariant	48
15.5 DP State Invariant	48
15.6 Why Proofs Help with Debugging	48
16 Current Progress and Final Reflection	50
16.1 Areas That Still Require Practice	50
16.2 How I Plan to Improve	50
16.3 Final Conclusion	50

1. Introduction and Development of My Approach

Before this course, I often treated programming as a direct translation from an idea to code. I would read a question, think of the first approach that seemed reasonable, and immediately begin implementing it. This worked for simple traversal questions, but it became unreliable as the questions became longer or involved more than one condition.

In medium-level problems, the main difficulty was usually not Python syntax. The harder part was recognising the structure of the question. It was not always clear whether a problem required a sliding window, a prefix sum, binary search, recursion, a stack, a heap, or a combination of techniques.

I gradually started following a more structured process:

1. Read the problem carefully and identify the exact input and output.
2. Examine the constraints before selecting an approach.
3. Think of the simplest correct method first.
4. Calculate its approximate time and space complexity.
5. Identify repeated calculations or repeated searches.
6. Consider whether previous information can be stored.
7. Select a suitable data structure or algorithm.
8. Test the solution on ordinary and boundary cases.
9. Explain why the algorithm is correct.

This process has made unfamiliar questions less overwhelming. Even when I cannot immediately solve a medium-level problem, I can usually divide it into smaller questions and understand which part is causing difficulty.

1.1 Understanding an Explanation Versus Producing a Solution

One important realisation was that understanding an explanation is not the same as producing the idea independently.

After reading a sliding-window or monotonic-stack solution, the algorithm may look straightforward. However, starting from only the problem statement and discovering that pattern is much harder.

I therefore try to separate two stages of learning:

- understanding why an existing solution works;
- learning to recognise when that technique should be applied.

When I use ChatGPT, I try to use it mainly for the first stage. I may ask for the base intuition, the repeated work in my current solution, or the meaning of a state or invariant. After that, I try to rewrite and trace the algorithm myself.

1.2 Correctness Is Not the Final Step

A program can produce correct answers for the examples and still fail during submission. A complete solution must consider

correctness, time complexity, space complexity, edge cases.

Common reasons for failure include:

- the algorithm is too slow for the maximum input size;

- the program creates too many temporary lists;
- recursion becomes too deep;
- an index moves outside the valid range;
- an empty stack or queue is accessed;
- shared data is modified unintentionally;
- an empty or one-element input is not handled.

I now consider complexity analysis and testing as part of the solution rather than something added after the program is finished.

2. Multiple Solutions and Complexity Analysis

One of the most useful lessons from the course is that a single problem can have several correct solutions. These solutions may differ in running time, memory usage, clarity, and ease of implementation.

2.1 Example: Contains Duplicate

Suppose the task is to determine whether a list contains a repeated value.

The most direct method compares every possible pair.

```
1 def contains_duplicate(nums):
2     for i in range(len(nums)):
3         for j in range(i + 1, len(nums)):
4             if nums[i] == nums[j]:
5                 return True
6
7     return False
```

For the first value, approximately $n - 1$ comparisons are performed. For the second value, $n - 2$ comparisons are performed, and so on. Therefore, the total number of comparisons is

$$(n - 1) + (n - 2) + \cdots + 2 + 1.$$

Using

$$1 + 2 + \cdots + (n - 1) = \frac{n(n - 1)}{2},$$

we obtain

$$T(n) = O(n^2).$$

The method is correct because every possible pair of positions is checked. However, for a large input, the number of comparisons becomes too high.

2.2 Sorting-Based Solution

A second method sorts the list and compares adjacent values.

```
1 def contains_duplicate(nums):
2     sorted_nums = sorted(nums)
3
4     for i in range(1, len(sorted_nums)):
5         if sorted_nums[i] == sorted_nums[i - 1]:
6             return True
7
8     return False
```

After sorting, equal values must appear next to each other. Therefore, checking adjacent positions is sufficient.

The sorting step takes approximately

$$O(n \log n)$$

time, and the traversal takes $O(n)$. Thus,

$$O(n \log n) + O(n) = O(n \log n).$$

2.3 Set-Based Solution

A third method uses a Python set.

```

1 def contains_duplicate(nums):
2     seen_numbers = set()
3
4     for number in nums:
5         if number in seen_numbers:
6             return True
7
8     seen_numbers.add(number)
9
10    return False

```

The average cost of a set membership check and insertion is $O(1)$. Since each value is processed once,

$$T(n) = O(n).$$

The set may store up to n values, so

$$S(n) = O(n).$$

Claim. *The set-based algorithm returns `True` if and only if a duplicate exists.*

Proof. Suppose the algorithm returns `True` for a value x . This happens only when x is already in the set. Since the set contains only earlier values, x must have appeared before.

Conversely, suppose a duplicate value x exists. Its first occurrence is inserted into the set. When a later occurrence is processed, the membership test finds x , so the algorithm returns `True`. $\square$

2.4 Time–Memory Trade-Off

This example shows a common trade-off:

$$\text{less execution time} \iff \text{additional memory}.$$

The set based solution is faster on average but stores previous values. The sorting solution requires sorting and creates a new list. The brute-force solution uses little extra memory but takes much longer.

2.5 Common Complexity Classes

Complexity	Meaning
$O(1)$	Constant amount of work
$O(\log n)$	Search space is repeatedly divided
$O(n)$	Every element is processed a fixed number of times
$O(n \log n)$	Efficient sorting or divide-and-conquer
$O(n^2)$	Every pair may be examined
$O(2^n)$	Every binary choice may be explored
$O(n!)$	Every possible ordering may be generated

The input constraints often suggest which complexity is expected:

- $n \leq 20$: exponential backtracking may be possible;
- $n \approx 10^3$: some $O(n^2)$ methods may work;
- $n \approx 10^5$: usually $O(n)$ or $O(n \log n)$;
- very large search space: often $O(\log n)$ or mathematical reasoning.

3. Array Traversal, Frequency Counting, and Prefix Techniques

Arrays were the starting point for many of the problems. The main challenge was deciding what information should be maintained during traversal.

3.1 Maximum Consecutive Ones

```
1 def find_max_consecutive_ones(nums):
2     current_count = 0
3     maximum_count = 0
4
5     for number in nums:
6         if number == 1:
7             current_count += 1
8         else:
9             current_count = 0
10
11         maximum_count = max(
12             maximum_count,
13             current_count
14         )
15
16     return maximum_count
```

Invariant. After processing a position, *current_count* stores the number of consecutive ones ending exactly at that position.

If the current value is 1, it extends the previous sequence. If the value is 0, no sequence of ones can end there, so the count becomes zero.

The maximum value observed is therefore the longest consecutive sequence.

$$T(n) = O(n), \quad S(n) = O(1).$$

3.2 Majority Element and Cancellation

The Boyer–Moore voting algorithm uses a cancellation argument.

```
1 def majority_element(nums):
2     candidate = None
3     count = 0
4
5     for number in nums:
6         if count == 0:
7             candidate = number
8
9         if number == candidate:
10             count += 1
11         else:
12             count -= 1
13
14     return candidate
```

Suppose the majority element appears more than $n/2$ times. Each different value can cancel one occurrence of the majority element. Since the majority appears more frequently than all other values combined, at least one majority occurrence remains uncanceled.

The algorithm simulates this cancellation process.

$$T(n) = O(n), \quad S(n) = O(1).$$

3.3 Kadane's Algorithm

Kadane's algorithm finds the maximum sum of a contiguous subarray.

```

1 def maximum_subarray(nums):
2     current_sum = nums[0]
3     maximum_sum = nums[0]
4
5     for i in range(1, len(nums)):
6         current_sum = max(
7             nums[i],
8             current_sum + nums[i]
9         )
10
11        maximum_sum = max(
12            maximum_sum,
13            current_sum
14        )
15
16    return maximum_sum

```

The recurrence is

$$\text{current_sum} = \max(\text{nums}[i], \text{current_sum} + \text{nums}[i]).$$

A maximum-sum subarray ending at index i either starts at i or extends the best subarray ending at $i - 1$.

Claim. After processing index i , *current_sum* is the maximum sum of any subarray ending at i .

Proof. Any contiguous subarray ending at i either contains index $i - 1$ or does not. If it does not, its sum is $\text{nums}[i]$. If it does, the best previous part is the maximum-sum subarray ending at $i - 1$.

The recurrence selects the better possibility. $\square$

3.4 Product of Array Except Self

My original approach repeatedly calculated the product of values before and after each position. It was correct but repeated many calculations.

The improved observation is

$$\text{answer}[i] = \left(\prod_{j=0}^{i-1} \text{nums}[j] \right) \left(\prod_{j=i+1}^{n-1} \text{nums}[j] \right).$$

```

1 def product_except_self(nums):
2     n = len(nums)
3
4     left_products = [1] * n
5     right_products = [1] * n
6     answer = []
7
8     for i in range(1, n):
9         left_products[i] = (
10             left_products[i - 1] * nums[i - 1]
11         )

```

```

12
13     for i in range(n - 2, -1, -1):
14         right_products[i] = (
15             right_products[i + 1] * nums[i + 1]
16         )
17
18     for i in range(n):
19         answer.append(
20             left_products[i] * right_products[i]
21         )
22
23     return answer

```

Claim. For every index i , $\text{left_products}[i]$ is the product of all values strictly before i .

Proof. For $i = 0$, there are no earlier values, so the product is 1.

Assume the result is correct for $i - 1$. Then

$$\text{left_products}[i] = \text{left_products}[i-1] \cdot \text{nums}[i-1].$$

Therefore, the product now includes exactly the values from index 0 to $i - 1$. The result follows by induction. $\square$

The same argument applies to the right products.

$$T(n) = O(n), \quad S(n) = O(n).$$

3.5 Prefix Sums

For an array a , define

$$P[i] = a[0] + a[1] + \cdots + a[i].$$

Then the sum from index l to r is

$$P[r] - P[l - 1]$$

for $l > 0$.

Proof. By definition,

$$P[r] = a[0] + \cdots + a[l - 1] + a[l] + \cdots + a[r],$$

and

$$P[l - 1] = a[0] + \cdots + a[l - 1].$$

Subtracting cancels all terms before l , leaving

$$a[l] + \cdots + a[r].$$

$\square$

For q range queries, prefix sums reduce the total time from approximately

$$O(nq)$$

to

$$O(n + q).$$

3.6 Prefix Remainders

If two prefix sums satisfy

$$P[j] \bmod k = P[i] \bmod k,$$

then

$$(P[j] - P[i]) \bmod k = 0.$$

Therefore, the subarray between the two positions has a sum divisible by k .

This turns an $O(n^2)$ subarray-checking problem into a frequency-counting problem.

4. Sliding Windows and Two-Pointer Methods

Sliding-window problems were among the first medium-level questions that I found difficult. The code may look short, but identifying the correct window condition is not always easy.

4.1 General Sliding-Window Structure

```
1 left = 0
2
3 for right in range(len(nums)):
4
5     # Add nums[right] to the current window
6
7     while window_is_invalid:
8
9         # Remove nums[left] from the window
10        left += 1
11
12    # Update the answer
```

Invariant. *After the shrinking loop finishes, the current window satisfies the required condition.*

4.2 Why the Complexity Is Linear

The right pointer moves forward n times. The left pointer also moves only forward and increases at most n times.

Therefore, the total number of pointer movements is at most

$$2n.$$

Thus,

$$T(n) = O(n).$$

This is an example of amortised analysis. One iteration may move the left pointer several times, but each element enters and leaves the window at most once.

4.3 Longest Substring Without Repeated Characters

```
1 def length_of_longest_substring(text):
2     character_count = {}
3     left = 0
4     longest_length = 0
5
6     for right in range(len(text)):
7         current_character = text[right]
8
9         character_count[current_character] = (
10             character_count.get(current_character, 0) + 1
11         )
12
13         while character_count[current_character] > 1:
14             left_character = text[left]
15             character_count[left_character] -= 1
16             left += 1
17
18         current_length = right - left + 1
```

```

19     longest_length = max(
20         longest_length,
21         current_length
22     )
23
24     return longest_length

```

Claim. *For each right endpoint, the algorithm finds the longest valid substring ending at that endpoint.*

Proof. When a duplicate is introduced, the left pointer moves until that duplicate is removed. It stops at the first position where the window becomes valid.

Any earlier left position would still contain a repeated character. Therefore, the current left position gives the longest valid substring ending at the current right position. $\square$

4.4 Fixed-Size Sliding Windows

For a fixed-size window of length k , the sum can be updated using

$$\text{new sum} = \text{old sum} - \text{outgoing value} + \text{incoming value}.$$

A direct method that recalculates every window sum takes

$$O(nk)$$

time. Maintaining the sum reduces the complexity to

$$O(n).$$

4.5 Two Pointers on Sorted Data

Suppose

$$a[l] + a[r] < \text{target}.$$

Moving the right pointer left would reduce the sum further. Therefore, the left pointer must move right.

Similarly, if

$$a[l] + a[r] > \text{target},$$

the right pointer must move left.

The sorted order justifies discarding one set of possibilities after each comparison.

4.6 Three Sum

The 3 Sum problem combines sorting and two pointers.

After sorting, one index is fixed, and the remaining two values are searched using left and right pointers. Sorting requires

$$O(n \log n).$$

The two-pointer search is $O(n)$ for every fixed index, giving

$$O(n^2)$$

overall. This is better than checking every triple, which takes

$$O(n^3).$$

5. Sorting Algorithms and Binary Search

Sorting algorithms were useful because they showed how different methods can produce the same output while having different performance.

5.1 Bubble Sort

```
1 def bubble_sort(nums):
2     n = len(nums)
3
4     for i in range(n):
5         swapped = False
6
7         for j in range(0, n - i - 1):
8             if nums[j] > nums[j + 1]:
9                 nums[j], nums[j + 1] = (
10                     nums[j + 1],
11                     nums[j]
12                 )
13                 swapped = True
14
15         if not swapped:
16             break
17
18     return nums
```

Invariant. After k complete passes, the final k positions contain the k largest values in sorted order.

The worst-case number of comparisons is

$$(n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2}.$$

Therefore,

$$T(n) = O(n^2).$$

5.2 Selection Sort

Selection sort repeatedly finds the minimum value in the unsorted portion.

Invariant. Before iteration i , the first i positions contain the i smallest values in sorted order.

It also takes $O(n^2)$ time but generally performs fewer swaps than bubble sort.

5.3 Insertion Sort

```
1 def insertion_sort(nums):
2     for i in range(1, len(nums)):
3         current_value = nums[i]
4         j = i - 1
5
6         while j >= 0 and nums[j] > current_value:
7             nums[j + 1] = nums[j]
8             j -= 1
9
10        nums[j + 1] = current_value
11
```

```
12     return nums
```

Invariant. *At the beginning of iteration i , the portion from index 0 to $i - 1$ is sorted.*

The worst-case complexity is

$$O(n^2),$$

but the algorithm performs well for nearly sorted data.

5.4 Merge Sort

```
1 def merge_sort(nums):
2     if len(nums) <= 1:
3         return nums
4
5     middle = len(nums) // 2
6
7     left_half = merge_sort(nums[:middle])
8     right_half = merge_sort(nums[middle:])
9
10    return merge(left_half, right_half)
11
12
13 def merge(left_half, right_half):
14     merged = []
15
16     left_index = 0
17     right_index = 0
18
19     while (
20         left_index < len(left_half)
21         and right_index < len(right_half)
22     ):
23         if (
24             left_half[left_index]
25             <= right_half[right_index]
26         ):
27             merged.append(left_half[left_index])
28             left_index += 1
29         else:
30             merged.append(right_half[right_index])
31             right_index += 1
32
33     merged.extend(left_half[left_index:])
34     merged.extend(right_half[right_index:])
35
36     return merged
```

The recurrence is

$$T(n) = 2T(n/2) + O(n).$$

There are approximately $\log_2 n$ levels, and each level processes n elements. Therefore,

$$T(n) = O(n \log n).$$

Claim. *Merging two sorted lists produces one sorted combined list.*

Proof. At each step, the algorithm compares the smallest unmerged value from each list. Since both lists are sorted, the smaller current value must be the smallest remaining value overall.

Selecting it preserves sorted order. Repeating the process produces a fully sorted result. $\square$

5.5 Binary Search

```
1 def binary_search(nums, target):
2     left = 0
3     right = len(nums) - 1
4
5     while left <= right:
6         middle = left + (right - left) // 2
7
8         if nums[middle] == target:
9             return middle
10
11        if nums[middle] < target:
12            left = middle + 1
13        else:
14            right = middle - 1
15
16    return -1
```

Invariant. *If the target exists, it remains inside the current search interval.*

After k steps, the search-space size is approximately

$$\frac{n}{2^k}.$$

Setting this equal to 1 gives

$$k = \log_2 n.$$

Thus,

$$T(n) = O(\log n).$$

5.6 Binary Search on the Answer

This technique searches a numerical answer range instead of stored indices.

The required condition must be monotonic, such as

False, False, False, True, True, True.

For example, if a shipping capacity x is sufficient, every capacity larger than x is also sufficient.

The difficult part is usually defining the feasibility function and proving that the condition is monotonic.

6. Stacks, Queues, Deques, and Monotonic Structures

Stacks and queues use different ordering rules and are therefore suitable for different types of problems.

6.1 Valid Parentheses

A stack follows Last In, First Out order.

```
1 def is_valid_parentheses(text):
2     stack = []
3
4     matching = {
5         ')': '(',
6         ']': '[',
7         '}': '{'
8     }
9
10    for character in text:
11        if character in '([{':
12            stack.append(character)
13        else:
14            if len(stack) == 0:
15                return False
16
17            top_bracket = stack.pop()
18
19            if top_bracket != matching[character]:
20                return False
21
22    return len(stack) == 0
```

Claim. *The algorithm accepts exactly the correctly matched and nested bracket strings.*

Proof. Every opening bracket is pushed onto the stack. A closing bracket must match the most recently opened unmatched bracket, which is stored at the top.

If the stack is empty, no opening bracket exists. If the types do not match, the nesting is invalid.

At the end, the stack must be empty; otherwise, some opening brackets remain unclosed. $\square$

6.2 Queues

A queue follows First In, First Out order.

Python's `collections.deque` supports efficient insertion at the back and removal from the front.

Queues are useful for:

- level-order tree traversal;
- breadth-first search;
- recent-request tracking;
- processing elements in discovery order.

6.3 Monotonic Stack

The Daily Temperatures problem stores unresolved indices in a decreasing stack.

```
1 def daily_temperatures(temperatures):
2     answer = [0] * len(temperatures)
```

```
3 stack = []
4
5 for current_index in range(len(temperatures)):
6
7     while (
8         stack
9         and temperatures[current_index]
10        > temperatures[stack[-1]]
11    ):
12        previous_index = stack.pop()
13
14        answer[previous_index] = (
15            current_index - previous_index
16        )
17
18        stack.append(current_index)
19
20 return answer
```

Each index is pushed once and popped at most once. Therefore, the total number of stack operations is at most $2n$.

$$T(n) = O(n).$$

6.4 Monotonic Deque

In the Sliding Window Maximum problem, a deque stores indices in decreasing order of their values.

```
1 from collections import deque
2
3 def max_sliding_window(nums, window_size):
4     result = []
5     indices = deque()
6
7     for right in range(len(nums)):
8
9         while (
10            indices
11            and indices[0] <= right - window_size
12        ):
13            indices.popleft()
14
15        while (
16            indices
17            and nums[indices[-1]] <= nums[right]
18        ):
19            indices.pop()
20
21        indices.append(right)
22
23        if right >= window_size - 1:
24            result.append(nums[indices[0]])
25
26    return result
```

Smaller values at the back are removed because a newer and larger value will remain useful for at least as long as they do.

Each index enters and leaves the deque at most once, so

$$T(n) = O(n).$$

7. Recursion and Backtracking

Recursion became easier when I started viewing each function call as solving a smaller version of the same problem.

A valid recursive solution requires:

1. a base case;
2. progress towards the base case;
3. a correct relation between the current problem and the smaller one.

7.1 Factorial and Induction

```
1 def factorial(number):  
2     if number == 0:  
3         return 1  
4  
5     return number * factorial(number - 1)
```

The definition is

$$n! = n(n-1)!$$

with

$$0! = 1.$$

Claim. *The recursive function returns $n!$ for every non-negative integer n .*

Proof. For $n = 0$, the function returns $1 = 0!$.

Assume that the function correctly returns $k!$. For $k + 1$, it returns

$$(k + 1) \times \text{factorial}(k).$$

Using the induction assumption,

$$(k + 1) \times k! = (k + 1)!.$$

Therefore, the function is correct for every non-negative integer. □

7.2 Recursion Depth

Every recursive call occupies stack memory. If the recursion depth is n , the auxiliary space is generally

$$O(n).$$

Python also has a recursion-depth limit. Therefore, a logically correct recursive method may still raise a `RecursionError`.

7.3 Backtracking

Backtracking makes a choice, explores it, and then reverses it.

```
1 def solve(...):  
2     if solution_is_complete:
```

```

3     save_solution()
4     return
5
6     for choice in possible_choices:
7         make_choice(choice)
8
9         solve(...)
10
11        undo_choice(choice)

```

The undo step is essential because each branch should begin from the state that existed before the choice.

7.4 Generating Subsets

```

1 def subsets(nums):
2     result = []
3     current_subset = []
4
5     def generate(index):
6         if index == len(nums):
7             result.append(current_subset[:])
8             return
9
10            current_subset.append(nums[index])
11            generate(index + 1)
12            current_subset.pop()
13
14            generate(index + 1)
15
16    generate(0)
17    return result

```

At each index, the element is either included or excluded. Therefore, the number of possible subsets is

$$2^n.$$

Claim. *The algorithm generates every subset exactly once.*

Proof. Every subset corresponds to a sequence of n include-or-exclude decisions. The algorithm explores both decisions at every index, so every possible sequence is visited.

Different decision sequences select different sets of positions. Therefore, every subset is generated exactly once. $\square$

The copy

```

1 current_subset[:]

```

is necessary because the same list is modified during later recursive calls.

7.5 Generate Parentheses and Pruning

For valid parentheses, the following conditions are maintained:

$$\text{closing count} \leq \text{opening count}$$

and

opening count $\leq n$.

If the closing count exceeds the opening count, that partial string can never become valid. Ending such branches early is called pruning.

8. Introduction to Dynamic Programming

Dynamic Programming has only been introduced at a basic level so far. The three problems I completed were:

- Climbing Stairs;
- Min Cost Climbing Stairs;
- House Robber.

These problems introduced the basic ideas of states, recurrence relations, base cases, overlapping subproblems, and tabulation.

8.1 Why Dynamic Programming Is Useful

Dynamic Programming is useful when a problem has:

1. **Overlapping subproblems:** the same smaller problem appears repeatedly.
2. **Optimal substructure:** the solution to a larger problem can be built from solutions to smaller problems.

A general DP process is:

1. Define what one state represents.
2. Write a recurrence connecting the state to earlier states.
3. Set the base cases.
4. Calculate states in a valid order.
5. Return the state representing the complete problem.

8.2 Memoisation and Tabulation

Memoisation starts with recursion and stores the result of every state. When a state is requested again, the saved result is reused.

Tabulation builds the answers iteratively, usually from smaller states to larger states.

The basic problems I solved used tabulation because the order of calculation was straightforward.

8.3 Climbing Stairs

At every move, one or two steps may be climbed.

Let

$$dp[i]$$

represent the number of ways to reach step i .

The final move to step i must come from either:

- step $i - 1$ using one step;
- step $i - 2$ using two steps.

Therefore,

$$dp[i] = dp[i - 1] + dp[i - 2].$$

```
1 def climb_stairs(n):
```

```

2  if n == 1:
3      return 1
4
5  if n == 2:
6      return 2
7
8  dp = [0] * (n + 1)
9
10 dp[1] = 1
11 dp[2] = 2
12
13 for i in range(3, n + 1):
14     one_step_before = dp[i - 1]
15     two_steps_before = dp[i - 2]
16
17     dp[i] = (
18         one_step_before
19         + two_steps_before
20     )
21
22 return dp[n]

```

Claim. For every i , $dp[i]$ equals the number of distinct ways to reach step i .

Proof. Any valid path to step i ends with either a one-step move from $i - 1$ or a two-step move from $i - 2$. These two groups are disjoint because their final moves are different.

Therefore, the total number of paths is

$$dp[i - 1] + dp[i - 2].$$

The base cases $dp[1] = 1$ and $dp[2] = 2$ are correct, so the recurrence constructs the correct answer for all later steps. $\square$

$$T(n) = O(n), \quad S(n) = O(n).$$

8.4 Min Cost Climbing Stairs

Let

$$dp[i]$$

represent the minimum cost required to reach position i .

Position i can be reached from:

- position $i - 1$, paying $\text{cost}[i - 1]$;
- position $i - 2$, paying $\text{cost}[i - 2]$.

Therefore,

$$dp[i] = \min(dp[i - 1] + \text{cost}[i - 1], dp[i - 2] + \text{cost}[i - 2]).$$

```

1 def min_cost_climbing_stairs(cost):
2     n = len(cost)
3
4     dp = [0] * (n + 1)

```

```

5
6 dp[0] = 0
7 dp[1] = 0
8
9 for i in range(2, n + 1):
10     cost_from_one_step = (
11         dp[i - 1] + cost[i - 1]
12     )
13
14     cost_from_two_steps = (
15         dp[i - 2] + cost[i - 2]
16     )
17
18     dp[i] = min(
19         cost_from_one_step,
20         cost_from_two_steps
21     )
22
23 return dp[n]

```

Claim. For each position i , $dp[i]$ stores the minimum possible cost required to reach that position.

Proof. The final move to i must come from either $i - 1$ or $i - 2$. There are no other possible final moves.

By assuming the stored values for those earlier positions are minimal, the minimum of the two possible final transitions gives the minimum cost to reach i . $\square$

8.5 House Robber

In this problem, adjacent houses cannot both be selected.

Let

$$dp[i]$$

represent the maximum amount that can be collected from houses 0 to i .

At house i , there are two choices:

1. skip house i , keeping $dp[i - 1]$;
2. rob house i , adding its value to $dp[i - 2]$.

Therefore,

$$dp[i] = \max(dp[i - 1], dp[i - 2] + \text{nums}[i]).$$

```

1 def rob(nums):
2     n = len(nums)
3
4     if n == 1:
5         return nums[0]
6
7     dp = [0] * n
8
9     dp[0] = nums[0]
10    dp[1] = max(nums[0], nums[1])
11
12    for i in range(2, n):
13        rob_current_house = (

```

```
14         dp[i - 2] + nums[i]
15     )
16
17     skip_current_house = dp[i - 1]
18
19     dp[i] = max(
20         rob_current_house,
21         skip_current_house
22     )
23
24     return dp[n - 1]
```

Claim. For every i , $dp[i]$ is the maximum amount obtainable from houses 0 through i without selecting adjacent houses.

Proof. Any valid solution for houses up to i either includes house i or does not.

If house i is excluded, the best amount is $dp[i - 1]$.

If house i is included, house $i - 1$ cannot be selected, so the best amount is

$$dp[i - 2] + \text{nums}[i].$$

Taking the maximum of these two complete possibilities gives the optimal answer. □

8.6 What I Learned from Basic DP

These problems showed me that the hardest part of DP is often defining the state clearly.

Once the meaning of $dp[i]$ is precise, the recurrence becomes easier to derive. I also learned that the base cases are part of the mathematical model, not only special programming conditions.

I have not yet covered advanced DP topics such as multidimensional states, subsequence DP, interval DP, or state compression. My current understanding is limited to basic one-dimensional recurrences.

9. Further Dynamic Programming: Subsequences, Strings, Stocks, and Grids

After doing the first few basic DP problems, I realised that Dynamic Programming is not just one fixed technique. The first problems mostly had a one-dimensional state like $dp[i]$, but the newer questions made the state much harder to identify. This was where I struggled more because the code usually follows after the state is clear, but choosing the state itself is the actual difficult part.

Earlier, I thought DP mostly meant storing previous answers. That is true, but it is not enough. For each problem, I had to ask:

- What does one state actually mean?
- What previous states are needed to calculate it?
- What are the base cases?
- In which order should the table be filled?
- Is the answer asking for a count, maximum, minimum, or possibility?

This made DP feel more mathematical than I expected. A small change in the meaning of the state can completely change the recurrence.

9.1 DP on Subsequences

Problems like Partition Equal Subset Sum, Target Sum, Last Stone Weight II, Coin Change, and Coin Change II all had a similar hidden idea: instead of only thinking about indices, I also had to think about a target value or sum.

For example, in subset-sum style problems, a state can be written as

$$dp[i][s],$$

where it represents whether it is possible to form sum s using the first i elements.

For Partition Equal Subset Sum, the main observation is that if the total sum is odd, equal partition is impossible. If it is even, the question becomes:

$$\text{Can we form sum } \frac{\text{total}}{2}?$$

A beginner friendly version I made:

```
1 def can_partition(nums):
2     total_sum = sum(nums)
3
4     if total_sum % 2 == 1:
5         return False
6
7     target = total_sum // 2
8     possible = [False] * (target + 1)
9
10    possible[0] = True
11
12    for number in nums:
13        for current_sum in range(target, number - 1, -1):
14            if possible[current_sum - number]:
15                possible[current_sum] = True
16
17    return possible[target]
```

The part that took time to understand was the reverse loop. If I looped forward, the same number could accidentally be used more than once in the same iteration. Looping backwards preserves the idea that every number is either taken once or not taken.

This was one of the first places where I realised that even a small loop direction can have a mathematical reason behind it.

9.2 Coin Change and Counting Versus Minimising

Coin Change and Coin Change II looked similar at first, but they are actually asking different things. Coin Change asks for the minimum number of coins, whereas Coin Change II asks for the number of combinations.

For minimum coins, the state can be:

$$dp[x] = \text{minimum coins needed to form amount } x.$$

```

1 def coin_change(coins, amount):
2     large_value = amount + 1
3     dp = [large_value] * (amount + 1)
4
5     dp[0] = 0
6
7     for current_amount in range(1, amount + 1):
8         for coin in coins:
9             if current_amount - coin >= 0:
10                dp[current_amount] = min(
11                    dp[current_amount],
12                    dp[current_amount - coin] + 1
13                )
14
15     if dp[amount] == large_value:
16         return -1
17
18     return dp[amount]
```

The main struggle here was understanding what exactly is being optimised. In some problems, $dp[i]$ stores a count; in others, it stores a minimum or maximum. The recurrence depends on that meaning. I learnt that before writing the code, I need to write in words what $dp[i]$ represents.

9.3 DP on Strings

String DP was a big jump in difficulty because the table usually depends on two indices instead of one. Problems such as Longest Common Subsequence, Longest Palindromic Subsequence, Distinct Subsequences, Delete Operation for Two Strings, Minimum Insertions to Make a String Palindrome, and Edit Distance all required comparing prefixes of strings.

For Longest Common Subsequence, the state is:

$$dp[i][j] = \text{length of the LCS of the first } i \text{ characters of text1 and first } j \text{ characters of text2.}$$

```

1 def longest_common_subsequence(text1, text2):
2     rows = len(text1)
3     cols = len(text2)
4
5     dp = []
6
```

```

7   for i in range(rows + 1):
8       current_row = [0] * (cols + 1)
9       dp.append(current_row)
10
11  for i in range(1, rows + 1):
12      for j in range(1, cols + 1):
13          if text1[i - 1] == text2[j - 1]:
14              dp[i][j] = 1 + dp[i - 1][j - 1]
15          else:
16              dp[i][j] = max(
17                  dp[i - 1][j],
18                  dp[i][j - 1]
19              )
20
21  return dp[rows][cols]

```

The intuition is that if the last characters match, they can be included in the subsequence. If they do not match, one of them must be skipped. This made the recurrence feel less random.

String DP was one of the topics where I had to use help to get the base intuition. Once I understood that every cell compares two prefixes, the table started making more sense.

9.4 Edit Distance and Thinking in Operations

Edit Distance was different because the recurrence is based on operations: insert, delete, or replace. Initially, this was confusing because I was trying to think only in terms of matching characters. The better way was to think:

$dp[i][j]$ = minimum operations needed to convert first i characters of word1 to first j characters of word2.

If the current characters match, no operation is needed. Otherwise, the answer comes from the minimum of insert, delete, and replace.

This topic showed me that DP can represent a process, not just a sum or count.

9.5 DP on Stocks

The stock problems were interesting because the state is not just the day. I also had to store whether I was holding a stock, whether I was in cooldown, or how many transactions were left.

For simple stock buying and selling with multiple transactions, the idea is to keep two states:

cash = best profit when not holding a stock,

hold = best profit when holding a stock.

```

1  def max_profit(prices):
2      cash = 0
3      hold = -prices[0]
4
5      for i in range(1, len(prices)):
6          price = prices[i]
7
8          new_cash = max(cash, hold + price)
9          new_hold = max(hold, cash - price)
10
11         cash = new_cash

```

```

12         hold = new_hold
13
14     return cash

```

This was a good example of manipulating an algorithm creatively. The same basic state idea can be changed for a transaction fee, cooldown, or limited number of transactions. The difficult part is not always the code; it is deciding what extra information the state needs.

9.6 LIS and Pattern-Based DP

Longest Increasing Subsequence introduced a different type of DP. Here,

$$dp[i]$$

can mean the length of the longest increasing subsequence ending at index i .

```

1 def length_of_lis(nums):
2     dp = [1] * len(nums)
3
4     for i in range(len(nums)):
5         for j in range(i):
6             if nums[j] < nums[i]:
7                 dp[i] = max(
8                     dp[i],
9                     dp[j] + 1
10                )
11
12     return max(dp)

```

The main idea is that to extend a subsequence ending at j , the value at j must be smaller than the value at i . This looked easy after seeing it, but it was not obvious from the start. It taught me that sometimes the state should describe what the solution ends with, not only how much of the array has been processed.

9.7 Grid DP and Matrix-Based Problems

Problems like Unique Paths, Minimum Path Sum, Triangle, Minimum Falling Path Sum, Maximal Square, and Count Square Submatrices introduced grid-based DP.

In these problems, each cell usually depends on neighbouring cells. For example, in Minimum Path Sum:

$$dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1]).$$

The intuition is simple once the movement rule is clear: to reach a cell, the last step must have come from one of the allowed previous cells.

Some grid DP problems were much harder, especially Cherry Pickup and Cherry Pickup II, because they require tracking two paths or two people at the same time. I understood the basic idea, but these problems still felt advanced because the state has more dimensions. This is one area where I still need more practice.

9.8 Main DP Learning

The biggest learning from these DP topics was that the code is usually the last step. The real work is defining the state.

I also learnt that the same basic DP idea can be stretched in many different ways:

- one dimensional DP for stairs and house robber;
- sum based DP for subsequences and coins;
- two dimensional DP for strings;
- state machine DP for stocks;
- grid DP for paths and matrices;
- interval style thinking for harder problems like Burst Balloons.

This made me understand why DP is considered difficult. The recurrence is not something to memorise blindly; it has to come from the meaning of the state.

10. Graph Algorithms, BFS, DFS, and Shortest Paths

Graphs were a very different shift from arrays and DP. In arrays, the neighbouring positions are usually obvious. In graphs, I first had to build the connections properly before even applying an algorithm.

The newer graph problems included Number of Provinces, Flood Fill, Rotting Oranges, 01 Matrix, Surrounded Regions, Number of Enclaves, Word Ladder, Course Schedule, Network Delay Time, Cheapest Flights Within K Stops, Path With Minimum Effort, and other shortest-path style problems.

10.1 DFS and Connected Components

DFS is useful when we want to explore everything connected to a starting node.

For Number of Provinces, the main idea is that every unvisited city starts a new component. DFS marks all cities connected to it.

```
1 def find_circle_num(is_connected):
2     number_of_cities = len(is_connected)
3     visited = [False] * number_of_cities
4
5     def dfs(city):
6         visited[city] = True
7
8         for next_city in range(number_of_cities):
9             if (
10                 is_connected[city][next_city] == 1
11                 and not visited[next_city]
12             ):
13                 dfs(next_city)
14
15     provinces = 0
16
17     for city in range(number_of_cities):
18         if not visited[city]:
19             provinces += 1
20             dfs(city)
21
22     return provinces
```

The invariant is that once DFS starts from a city, every city reachable from it will be marked visited. Therefore, counting the number of DFS starts gives the number of connected components.

One struggle here was remembering that graph traversal is not always about finding a path. Sometimes it is only about grouping connected nodes.

10.2 BFS and Level-by-Level Thinking

BFS is useful when the problem involves shortest distance in an unweighted graph or spreading level by level. Rotting Oranges was a good example because all rotten oranges spread at the same time.

This required multi-source BFS. Instead of starting from one source, all initial rotten oranges are placed in the queue.

```
1 from collections import deque
2
3 def oranges_rotting(grid):
4     rows = len(grid)
5     cols = len(grid[0])
```

```
6
7 queue = deque()
8 fresh_count = 0
9
10 for row in range(rows):
11     for col in range(cols):
12         if grid[row][col] == 2:
13             queue.append((row, col, 0))
14         elif grid[row][col] == 1:
15             fresh_count += 1
16
17 directions = [
18     (1, 0),
19     (-1, 0),
20     (0, 1),
21     (0, -1)
22 ]
23
24 minutes = 0
25
26 while queue:
27     row, col, time = queue.popleft()
28     minutes = max(minutes, time)
29
30     for row_change, col_change in directions:
31         new_row = row + row_change
32         new_col = col + col_change
33
34         if (
35             0 <= new_row < rows
36             and 0 <= new_col < cols
37             and grid[new_row][new_col] == 1
38         ):
39             grid[new_row][new_col] = 2
40             fresh_count -= 1
41             queue.append((new_row, new_col, time + 1))
42
43 if fresh_count != 0:
44     return -1
45
46 return minutes
```

The creative part here was realising that all starting rotten oranges must enter the queue at time 0. If I started BFS from only one rotten orange at a time, the answer would be wrong because the spread is simultaneous.

10.3 Grid Problems as Graphs

Problems like Flood Fill, Surrounded Regions, Number of Enclaves, and 01 Matrix are not presented as normal graphs, but the grid can be treated as a graph where each cell is connected to its neighbouring cells.

The main challenge was not DFS or BFS itself, but handling boundaries:

$$0 \leq row < rows, \quad 0 \leq col < cols.$$

Many wrong answers came from small boundary mistakes or from marking a cell as visited too late. I learnt that in BFS, it is usually safer to mark a cell when it is pushed into the queue, not when it is popped, because otherwise the same cell may be added multiple times.

10.4 Topological Sorting

Course Schedule introduced directed graphs and cycles. If a course depends on another course, this creates a directed edge.

Topological sorting works only when the graph has no cycle. Kahn's algorithm uses indegrees.

```
1 from collections import deque
2
3 def can_finish(num_courses, prerequisites):
4     graph = []
5
6     for i in range(num_courses):
7         graph.append([])
8
9     indegree = [0] * num_courses
10
11    for course, previous_course in prerequisites:
12        graph[previous_course].append(course)
13        indegree[course] += 1
14
15    queue = deque()
16
17    for course in range(num_courses):
18        if indegree[course] == 0:
19            queue.append(course)
20
21    completed_courses = 0
22
23    while queue:
24        course = queue.popleft()
25        completed_courses += 1
26
27        for next_course in graph[course]:
28            indegree[next_course] -= 1
29
30            if indegree[next_course] == 0:
31                queue.append(next_course)
32
33    return completed_courses == num_courses
```

The intuition is that a course with indegree 0 has no remaining prerequisite. Removing it may make other courses available. If some courses never become available, there must be a cycle.

This was one of the first times where the graph direction mattered a lot. If the edge was built backwards, the code still ran but gave wrong logic.

10.5 Shortest Path Algorithms

Shortest path problems were more difficult because BFS works only when all edges have equal weight. When weights are involved, Dijkstra's algorithm or other shortest-path algorithms are needed.

For Network Delay Time, the graph has weighted edges, so the smallest current distance is repeatedly selected using a heap.

```
1 import heapq
2
3 def network_delay_time(times, n, k):
4     graph = {}
5
6     for node in range(1, n + 1):
7         graph[node] = []
8
```

```
9     for start, end, time in times:
10         graph[start].append((end, time))
11
12     distance = {}
13
14     for node in range(1, n + 1):
15         distance[node] = float('inf')
16
17     distance[k] = 0
18     heap = [(0, k)]
19
20     while heap:
21         current_time, node = heapq.heappop(heap)
22
23         if current_time > distance[node]:
24             continue
25
26         for next_node, edge_time in graph[node]:
27             new_time = current_time + edge_time
28
29             if new_time < distance[next_node]:
30                 distance[next_node] = new_time
31                 heapq.heappush(
32                     heap,
33                     (new_time, next_node)
34                 )
35
36     answer = max(distance.values())
37
38     if answer == float('inf'):
39         return -1
40
41     return answer
```

The invariant is that when a node is processed with its smallest known distance, that distance is the best found so far. The heap allows the currently smallest tentative distance to be selected efficiently.

10.6 Manipulating Shortest Path Algorithms

Some problems required modifying the basic shortest-path idea. For example, Cheapest Flights Within K Stops cannot be solved by only storing the cheapest cost to each city, because the number of stops also matters. A slightly more expensive path with fewer stops may still be useful later.

This taught me that sometimes the node alone is not enough state. The state may need to include:

(city, stops used)

or

(row, col, effort)

depending on the problem.

This was one of the more creative parts of the new topics. The algorithm is not completely new, but it has to be reshaped to fit the problem's restrictions.

10.7 Floyd–Warshall

Floyd–Warshall was introduced as an all-pairs shortest path algorithm. The idea is to gradually allow more intermediate nodes.

The recurrence is:

$$dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j]) .$$

The interpretation is that the shortest path from i to j either does not use k , or it uses k as an intermediate node.

Its time complexity is

$$O(n^3).$$

This is not suitable for very large graphs, but it is useful when the graph is small and distances between every pair of nodes are needed.

11. Union-Find, MST Ideas, and Algorithm Adaptation

Union-Find was one of the newer topics where the actual code was not very long, but the way it is applied changes a lot from problem to problem.

It is useful for maintaining groups of connected components while edges or relationships are added.

11.1 Basic Union-Find

Union-Find supports two main operations:

- **find**: identify the parent or representative of a node;
- **union**: merge two components.

A standard implementation is:

```
1 class DSU:
2     def __init__(self, size):
3         self.parent = []
4         self.rank = []
5
6         for i in range(size):
7             self.parent.append(i)
8             self.rank.append(0)
9
10    def find(self, node):
11        if self.parent[node] != node:
12            self.parent[node] = self.find(
13                self.parent[node]
14            )
15
16        return self.parent[node]
17
18    def union(self, node1, node2):
19        parent1 = self.find(node1)
20        parent2 = self.find(node2)
21
22        if parent1 == parent2:
23            return False
24
25        if self.rank[parent1] < self.rank[parent2]:
26            self.parent[parent1] = parent2
27        elif self.rank[parent1] > self.rank[parent2]:
28            self.parent[parent2] = parent1
29        else:
30            self.parent[parent2] = parent1
31            self.rank[parent1] += 1
32
33        return True
```

Path compression makes future searches faster by directly connecting nodes to their representative. Union by rank keeps the tree shallow.

11.2 Connected Components and Network Problems

Number of Operations to Make Network Connected is a natural Union-Find problem. Every cable connects two computers, which means their components can be merged.

The important observation is that to connect n computers, at least

$$n - 1$$

edges are needed. If fewer edges exist, it is impossible.

After processing all edges, if there are c components, the number of operations required is

$$c - 1.$$

This made the problem feel much simpler than trying to manually build all connections.

11.3 Accounts Merge

Accounts Merge showed me that Union-Find is not only for numbered graph nodes. Emails can also be treated as nodes. If two emails appear in the same account, they belong to the same person and can be unioned.

This was a good example of creatively changing the meaning of a node. The algorithm is the same, but instead of cities or computers, the components are groups of emails.

11.4 Most Stones Removed

In Most Stones Removed with Same Row or Column, stones are connected if they share a row or column. At first, it was not obvious why Union-Find applies here. The useful observation is:

$$\text{stones removable} = \text{total stones} - \text{number of connected components}.$$

Within each connected component, all but one stone can be removed. This was a good example of the answer coming from a component count rather than from direct simulation.

11.5 Kruskal's and Prim's Algorithms

Kruskal's and Prim's algorithms were introduced for Minimum Spanning Tree ideas.

Kruskal's algorithm sorts edges by weight and adds them if they do not create a cycle. Union-Find is used to check whether adding an edge connects two already connected nodes.

Prim's algorithm grows the tree from a starting node by repeatedly choosing the smallest edge that connects the current tree to a new node.

The difference I understood is:

- Kruskal thinks in terms of edges globally;
- Prim thinks in terms of expanding from a connected set of nodes.

11.6 Learning to Manipulate Algorithms

The biggest learning from graphs and Union-Find was that algorithms are often templates, not fixed answers.

For example:

- BFS can become multi-source BFS in Rotting Oranges.
- BFS can store distance in 01 Matrix.
- Dijkstra can store extra state when stops or effort matter.
- Topological sort can detect whether all courses are possible.
- Union-Find can group computers, emails, stones, or island cells.

- Kruskal uses Union-Find to avoid cycles.

This is where I had to think more creatively. Earlier, I used to look for the exact algorithm name. Now I am trying to ask what the algorithm needs to store and how the state should be changed for the problem.

It was also easy to make small but serious mistakes in these topics:

- building graph edges in the wrong direction;
- forgetting to mark visited cells;
- marking visited too late in BFS;
- using BFS when weighted edges require Dijkstra;
- storing only the node when extra state is needed;
- not compressing paths in Union-Find;
- confusing components with individual nodes.

These mistakes were frustrating, but they were also useful because they showed me that the algorithm and representation must match the problem exactly.

12. Binary Trees and Binary Search Trees

Trees made recursion feel more natural because every subtree is another smaller tree.

12.1 Tree Traversals

The main depth-first traversal orders are:

Preorder: Root, Left, Right,

Inorder: Left, Root, Right,

Postorder: Left, Right, Root.

```

1 def inorder_traversal(root):
2     result = []
3
4     def traverse(node):
5         if node is None:
6             return
7
8         traverse(node.left)
9         result.append(node.val)
10        traverse(node.right)
11
12    traverse(root)
13    return result

```

Every node is visited once:

$$T(n) = O(n).$$

The recursion stack uses

$$O(h)$$

space, where h is the tree height.

12.2 Maximum Depth

```

1 def max_depth(root):
2     if root is None:
3         return 0
4
5     left_depth = max_depth(root.left)
6     right_depth = max_depth(root.right)
7
8     return 1 + max(left_depth, right_depth)

```

The recurrence is

$$\text{depth}(r) = 1 + \max(\text{depth}(r_{\text{left}}), \text{depth}(r_{\text{right}})).$$

Claim. *The function returns the length of the longest root-to-leaf path.*

Proof. Every root-to-leaf path continues through either the left subtree or the right subtree. The longest path through the left has length

$$1 + \text{depth}(r_{\text{left}}),$$

and the longest path through the right has length

$$1 + \text{depth}(r_{\text{right}}).$$

Taking the maximum gives the longest path. □

12.3 Level-Order Traversal

```

1 from collections import deque
2
3 def level_order(root):
4     if root is None:
5         return []
6
7     result = []
8     nodes = deque([root])
9
10    while nodes:
11        level_size = len(nodes)
12        current_level = []
13
14        for i in range(level_size):
15            node = nodes.popleft()
16            current_level.append(node.val)
17
18            if node.left is not None:
19                nodes.append(node.left)
20
21            if node.right is not None:
22                nodes.append(node.right)
23
24        result.append(current_level)
25
26    return result

```

The queue ensures that all nodes on one level are processed before nodes on the next level.

12.4 Binary Search Trees

A Binary Search Tree satisfies

$$x < r \quad \text{for every } x \text{ in the left subtree,}$$

and

$$x > r \quad \text{for every } x \text{ in the right subtree.}$$

```

1 def search_bst(root, target):
2     current_node = root
3
4     while current_node is not None:
5

```

```
6     if current_node.val == target:
7         return current_node
8
9     if target < current_node.val:
10         current_node = current_node.left
11     else:
12         current_node = current_node.right
13
14     return None
```

The running time is

$$O(h).$$

For a balanced BST,

$$h = O(\log n),$$

while for a skewed BST,

$$h = O(n).$$

12.5 Why Inorder Traversal Is Sorted

Claim. *The inorder traversal of a Binary Search Tree produces the values in increasing order.*

Proof. Every value in the left subtree is smaller than the root, while every value in the right subtree is larger.

The inorder traversal visits the sorted left subtree, then the root, then the sorted right subtree. Therefore, the complete result is sorted. $\square$

13. Greedy Algorithms and Heaps

Greedy algorithms make the best immediate choice. Their implementations may be short, but the local decision must be proved safe.

13.1 Assigning Cookies

The strategy is:

1. sort the child requirements;
2. sort the cookie sizes;
3. give the smallest sufficient cookie to the least demanding child.

```
1 def find_content_children(requirements, cookies):
2     requirements.sort()
3     cookies.sort()
4
5     child_index = 0
6     cookie_index = 0
7
8     while (
9         child_index < len(requirements)
10        and cookie_index < len(cookies)
11    ):
12        if cookies[cookie_index] >= requirements[child_index]:
13            child_index += 1
14
15        cookie_index += 1
16
17    return child_index
```

Using a larger cookie than necessary may reduce the options available for a more demanding child. The smallest sufficient cookie preserves larger cookies.

13.2 Interval Scheduling

Suppose the aim is to choose the maximum number of non-overlapping intervals. The greedy method selects the interval that finishes earliest.

Claim. *There exists an optimal solution whose first chosen interval is the earliest-finishing interval.*

Proof. Let G be the earliest-finishing interval. Consider an optimal solution whose first interval is O . If $O = G$, the claim holds.

Otherwise, replace O with G . Since G finishes no later than O , every interval that was compatible after O is also compatible after G .

Thus, the replacement does not reduce the number of selected intervals. $\square$

13.3 Jump Game

The algorithm maintains the farthest reachable index.

At index i ,

$$\text{farthest} = \max(\text{farthest}, i + \text{nums}[i]).$$

If i is greater than the farthest reachable index, then index i cannot be reached.

13.4 Heaps

Python's `heapq` module implements a min-heap.

Typical costs are:

$$\text{minimum access} = O(1),$$

$$\text{insertion} = O(\log n),$$

$$\text{removal} = O(\log n).$$

13.5 Finding the k -th Largest Element

```
1 import heapq
2
3 def find_kth_largest(nums, k):
4     heap = []
5
6     for number in nums:
7         heapq.heappush(heap, number)
8
9         if len(heap) > k:
10             heapq.heappop(heap)
11
12     return heap[0]
```

Invariant. *After processing any prefix, the heap contains the largest k values from that prefix, or all values if fewer than k values have been processed.*

At the end, the smallest value in this heap is the k -th largest overall.

$$T(n) = O(n \log k), \quad S(n) = O(k).$$

14. Implementation Errors, Time Limits, and Memory

Some of the most useful lessons came from programs that failed during submission.

14.1 Index Errors

For a list of length n , the valid indices are

$$0, 1, \dots, n - 1.$$

The following loop is incorrect:

```
1 for i in range(len(nums) + 1):  
2     print(nums[i])
```

The final value of i is `len(nums)`, which is outside the valid range.

14.2 Negative Indices

Python allows negative indices.

```
1 last_value = nums[-1]
```

This is convenient, but it can hide pointer mistakes. If an index accidentally becomes -1 , Python may access the final element instead of raising an immediate error.

14.3 Empty Containers

Calling `pop()` on an empty list or `popleft()` on an empty deque causes an error.

```
1 if stack:  
2     top_value = stack.pop()
```

14.4 None in Trees

Before accessing a node's value or children, the program must verify that the node is not `None`.

```
1 if node is None:  
2     return
```

14.5 Python List Slicing

A slice creates a new list.

```
1 part = nums[start:end]
```

If the slice contains m values, it requires

$$O(m)$$

time and memory.

Repeated slicing inside a loop can therefore make a solution much slower.

14.6 List Membership

Checking membership in a list may require scanning the complete list.

```
1 if number in nums:  
2     ...
```

The worst-case cost is

$$O(n).$$

If performed inside another loop, the total can become $O(n^2)$.

14.7 Copying and Shared Mutation

Passing a list to a function does not automatically create a copy.

```
1 def change_list(nums):  
2     nums.append(10)
```

This modifies the original list.

An independent copy can be created using

```
1 copied_nums = nums[:]
```

but copying a list of size n requires $O(n)$ time and memory.

14.8 Time Limit Exceeded

Common causes include:

- nested loops over large inputs;
- repeated linear searches;
- recalculating the same information;
- unnecessary sorting;
- repeated slicing;
- recursive exploration without pruning;
- using the wrong data structure.

A Time Limit Exceeded result often means that the overall approach must change.

14.9 Memory Limit Exceeded

Common causes include:

- very large lists or matrices;
- storing unnecessary intermediate results;
- repeated list copies;
- deep recursion;
- retaining data that is no longer needed.

15. Correctness Proofs and Invariants

A loop invariant is a statement that remains true during an algorithm.

A standard proof has three parts:

1. **Initialisation:** the invariant is true before the first iteration.
2. **Maintenance:** one iteration preserves the invariant.
3. **Termination:** when the loop ends, the invariant implies the required answer.

15.1 Insertion Sort Invariant

Before iteration i , the prefix from index 0 to $i - 1$ is sorted.

Initially, the prefix contains one value and is therefore sorted. During each iteration, the current value is inserted into the correct position. When the algorithm ends, the sorted prefix is the complete list.

15.2 Binary Search Invariant

If the target exists, it remains inside the current search interval.

Initially, the interval is the full list. At each step, sorted order proves that one half cannot contain the target. When the interval becomes empty, the target does not exist.

15.3 Sliding-Window Invariant

After shrinking, the current window satisfies the required condition.

This invariant helps determine when the left pointer should move and when the answer may be updated.

15.4 Heap Invariant

For the k -th largest problem, the heap stores the largest k values seen so far.

The invariant directly explains why the heap root is the final answer.

15.5 DP State Invariant

For a tabulation-based DP algorithm, the usual invariant is:

Before calculating state i , all previous states required by its recurrence have already been calculated correctly.

This explains why the order of filling the DP table matters.

15.6 Why Proofs Help with Debugging

Correctness proofs are also useful during debugging. When a program gives the wrong answer, I can ask:

- Did the binary-search interval still contain every possible answer?
- Was the sliding window valid when the answer was updated?
- Did the stack remain monotonic?
- Was the recursive state restored after backtracking?
- Did the heap still contain the correct candidates?
- Did the DP state represent what I intended?

This identifies which logical property failed rather than only checking random lines of code.

16. Current Progress and Final Reflection

I have become more comfortable with array traversal, frequency counting, prefix sums, standard binary search, basic recursion, elementary sorting, and simple stack problems.

After adding the newer topics, I have also started understanding more advanced patterns such as subsequence DP, string DP, stock DP, grid DP, BFS, DFS, topological sorting, shortest paths, and Union-Find. I would not say I have mastered these topics yet. For many of them, especially graph and DP medium questions, I still need time to identify the right state or representation. However, I am now better at reading a problem and asking what kind of information needs to be stored.

I can understand sliding windows, monotonic stacks, heaps, backtracking, binary search on the answer, DP recurrences, and graph traversals when the main idea is explained. However, I still find it difficult to independently identify these patterns in unfamiliar medium-level questions.

16.1 Areas That Still Require Practice

The main areas I am continuing to work on are:

- recognising the correct sliding-window condition;
- defining recursive and DP states clearly;
- deciding whether a DP problem needs one index, two indices, or extra state;
- understanding when a grid should be treated like a graph;
- choosing between DFS, BFS, Dijkstra, and Union-Find;
- building graph edges in the correct direction;
- proving that a greedy choice is safe;
- identifying monotonicity for binary search on the answer;
- avoiding unnecessary copying and repeated work;
- combining multiple techniques in one question.

16.2 How I Plan to Improve

A useful practice method for me is:

1. attempt the question independently for a fixed amount of time;
2. write a brute-force solution even if it is inefficient;
3. identify the repeated work;
4. request only a hint or intuition when necessary;
5. implement the improved approach independently;
6. trace the algorithm manually;
7. write the complexity and invariant;
8. solve a similar question without assistance.

This helps turn external guidance into actual learning.

16.3 Final Conclusion

The main lesson from the course so far is that solving a programming problem is not only about obtaining the correct output.

A complete solution should answer:

- Why does the method work?
- What information is maintained?

- What repeated work has been removed?
- How does the running time grow?
- How much memory is used?
- Which edge cases may cause failure?

Some of the most interesting algorithms were those where a small mathematical observation produced a large improvement. Prefix-sum cancellation, equal remainders, sliding-window amortisation, binary-search monotonicity, monotonic-stack behaviour, recursion induction, greedy exchange arguments, and DP recurrences all showed that mathematical reasoning directly supports efficient code.

Using ChatGPT for initial intuition has helped me approach questions that would otherwise feel inaccessible. However, I have also learned that reading a solution is much easier than producing one. I therefore try to use explanations as a starting point and then rewrite, trace, test, and justify the algorithm myself.

I am still developing confidence with medium-level problems, especially in the newer DP and graph topics. However, my approach has become more structured. I now compare multiple solutions, pay attention to constraints, analyse time and memory, define states and invariants, and think about edge cases before submitting code.

The newer topics also taught me that learning algorithms is not only about remembering their names. A lot of problem solving comes from creatively manipulating a known algorithm to fit the situation. BFS, Dijkstra, DP, and Union-Find all appeared in different forms depending on the problem. This change in thinking has been the most important outcome of the course so far.